Industrial Security (Bachelor)

Solution Sheet – Section 02

ICS and SCADA Fundamentals

How to use this sheet

Each box gives a model answer suitable for a TA briefing. Open-ended exercises (2.4, 2.6, 2.7) include rubric hints in brackets at the end of the solution – students do not have to match the model answer word for word, but the rubric items must be present.

Solution

Exercise 2.1 – Worst-Case Scan-Cycle Latency. The worst case stacks: the sensor edge is missed by the current cycle, so we wait one full cycle for the next input-read phase, then another cycle for the program to act on the new value, then the output card delay. Input filter: 3 ms. One missed cycle plus the following execute/update cycle: 12 + 12 = 24 ms. Output card: 2 ms. Total worst-case latency: 3 + 24 + 2 = 29 ms. The budget of 40 ms is met with 11 ms of margin. To shave latency, configure the input event on a hardware interrupt OB (OB40-family on S7-1500) so the reject does not have to wait for the cyclic scan – or use an HSC channel. To make it predictably worse, add a chatty OPC UA poll or large data block to the cyclic OB so housekeeping time pushes the worst-case cycle nearer to the watchdog.

Solution

Exercise 2.2 – Read This Ladder Rung. (1) On Start press, `X1`=1, `X2`=1 (Stop NC unpressed), `X3`=1 (tank not full): `Y1` energises. From the next scan on, the `Y1` seal-in keeps the rung true even after `X1` drops back to 0, so the pump runs continuously. (2) When the tank reaches the high-level switch, `X3` goes to 0, breaking the series path; `Y1` de-energises and the seal-in collapses on the same cycle. The pump stops automatically – this is the intended high-level shutoff. (3) Cutting the Stop wire makes `X2` read as 0 (no current through the NC contact's input circuit). The NC inversion makes `]/[` evaluate to 1, so the rung still works – which is the *wrong* failure mode: a Stop button whose wire is cut can no longer stop the pump. Stop must be wired so a broken wire *de-energises* the output (fail-safe). Most plants therefore wire Stop as a hard-wired series contact in the output circuit *outside* the PLC, not as a soft input.

Solution

Exercise 2.3 – Why Isolated SIS. (1) Two technical reasons: (a) *Common-cause failure*: any firmware bug, runtime fault, memory corruption, or malicious logic change in the shared M580 would defeat both the control *and* the protection function in a single event. (b) *Diversity and SIL certification*: a SIL 3 demand mode requires proven hardware reliability figures (PFD_{avg}) and a certified development process; a general-purpose BPCS controller is not certified for that and its firmware change cadence is incompatible with SIS recertification cycles. A typical SIS uses TMR (triple modular redundant) voting, which the M580 does not provide. (2) Regulatory: IEC 61511 (the process-industry application of IEC 61508) mandates that the SIS be "separate and independent" from the BPCS, and that any shared element be justified and analysed. EN 61511 is the EU-harmonised version; US sites often cite ISA 84.00.01. (3) TRITON compromised a Schneider Triconex by writing safety logic over the proprietary TriStation protocol. With shared BPCS+SIS on one controller, the attacker would need only one foothold to disable both control and safety. With a separate Triconex on its own engineering network, the

attacker must additionally pivot into the SIS engineering path *and* bypass the physical keyswitch – a much higher bar.

Solution

Exercise 2.4 – Windows XP HMI. A defensible top-five (any reasonable five; rubric items in brackets): (a) Put the HMI behind a small dedicated firewall (e.g. Fortinet FortiGate Rugged 60F or Phoenix Contact mGuard) with a strict allow-list: only the PLC IPs on Modbus/S7 ports outbound, only the operator-VLAN inbound on RDP from named jump host. (b) Disable every Windows service that is not strictly needed (Server, Workstation, Browser, Messenger); remove network printer sharing. (c) Apply application allow-listing with McAfee Application Control (formerly Solidcore) or Microsoft AppLocker configured in enforce mode – XP cannot get patches, so we stop unknown binaries from executing instead. (d) Remove all USB-mass-storage drivers or enforce DeviceLock; the engineering laptop uses a one-way file diode (e.g. Owl) or a sanitised USB station. (e) Move the HMI off the office domain and onto a local workgroup with unique local credentials; back the HMI image up nightly to an offline target so a re-image is one hour, not one week. The cheapest per-euro is the firewall plus allow-list (one rule set removes 95 % of the internet-exposed attack surface). Reject: "install antivirus" – modern AV signatures no longer ship for XP, performance is poor, and most XP-era exploits live below the AV hook anyway. [Rubric: at least one network control, at least one host-hardening control, at least one backup/recovery control, named products, one explicit rejection with reason.]

Solution

Exercise 2.5 – OPC UA on Pump P-101. (1) The engineer needs: routable IP reachability to the sensor on TCP port 4840 (default OPC UA binary); an OPC UA client such as UAExpert; the server's endpoint URL; if the server enforces a security policy other than `None`, an X.509 client certificate trusted by the server (typically pushed via the GDS or copied into the server's `trusted` folder); a valid user token (anonymous, username/password, or X.509 user). (2) The attacker needs the same three things – reachability, a client, and the endpoint. In a typical out-of-the-box deployment, sensor vendors ship with `SecurityPolicy=None` and `Anonymous` access enabled by default, so the attacker needs nothing extra beyond a network sniff to learn the endpoint URL. There is effectively no authentication gap between the two parties. (3) Disable the `None` security policy on the server, require `Basic256Sha256` with mutual certificate trust, and create a non-anonymous user token for the engineer. The engineer's UAExpert still works once her certificate is approved; the attacker now needs a trusted client certificate *and* valid credentials.

Solution

Exercise 2.6 – Historian for Billing. Three controls you would add for the billing historian, but not for a research-only one: (a) *Append-only write path with cryptographic auditing*: enable PI Audit Trail (PI Server feature) plus tamper-evident logging via the PI System Auditing module, with logs shipped off-box to a SIEM such as Splunk. (Lives in the PI Server and the SIEM.) (b) *Time integrity*: harden the historian's NTP source (use an internal stratum-1 GPS clock from Meinberg or Microsemi rather than public pool servers); enable PI's secure-time feature so out-of-order timestamps are flagged. (Lives in the OS and the PI Server.) (c) *Read-only, signed export to the billing system*: replace the PI OLEDB pull with a one-way PI-to-PI replication into a second "billing-PI" instance in a separate security zone, with PI Web API served over mutual TLS to the billing application; the billing system verifies row hashes recorded in PI Asset Framework. (Lives in PI AF, the network DMZ, and the billing system.) For a research

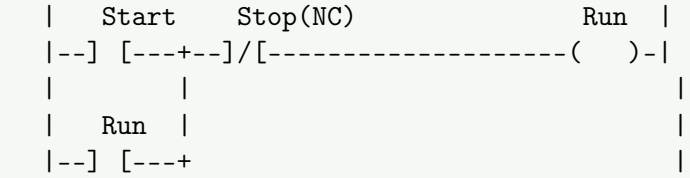
historian you do not care: scientists rerun their analytics, can fix bad timestamps, and no money rides on a single value. If the regulator can audit unannounced, also add immutable retention of the raw 1 s tag values for the entire billing-relevant period (typically 6+ years), so the auditor can recompute the totals. [Rubric: each control names where it lives and one concrete product or feature; the regulator add-on is about retention/non-repudiation, not just "add more logging".]

Solution

Exercise 2.7 – Open-Book: S7-1500 Scan Times. Model answer (Siemens S7-1500 System Manual, current edition at time of writing, e.g. 12/2023, chapter "Cycle and reaction times"): The minimum cycle time configurable for the cyclic main OB (OB 1) is 1 ms (entered in the CPU's "Cycle" properties in TIA Portal). The default maximum cycle-time monitoring (watchdog) is 150 ms; configurable from 1 ms up to 6 000 ms. Numbers vary slightly between SKUs (a CPU 1518 has higher throughput than a 1511) but the configurable range is the same. Siemens exposes a minimum so the user can guarantee a fixed cycle for deterministic control loops (the CPU idles to fill the cycle); the maximum is a watchdog – if the program ever runs past it, the CPU goes to STOP rather than silently violating real-time guarantees. Free-running at maximum speed would make jitter unbounded and break downstream timing. [Rubric: numbers are in the right order of magnitude (min $\approx$ 1 ms, max $\leq$ 6 000 ms), the citation names a specific section/table and edition date, and the explanation mentions determinism plus the watchdog purpose. Exact figures from the most current manual are acceptable; the student must cite the edition they read.]

Solution

Exercise 2.8 – Misplaced Seal-In Bug. (1) The seal-in is around *Start itself*, not around the whole start/stop chain. Critically, the second rung does not use the output `Run` as the seal contact – it uses `Start` again, which is the pushbutton input that returns to 0 as soon as the operator lets go. There is therefore no latch at all: the rung is just `(Start OR Start) AND NOT Stop` = `Start AND NOT Stop`. (2) The operator presses Start: while the button is held, `Run` energises and the motor turns. The moment she releases the button, `Run` drops out and the motor stops – behaving like a momentary pushbutton, not a latched start/stop. Pressing Stop does nothing additional, because the motor was never latched in the first place. (3) Corrected rung – the seal contact must be the output `Run` itself, in parallel with `Start`:



Now once `Run` energises for one scan, its own NO contact holds the rung true through the seal branch until Stop is pressed.

Solution

Exercise 2.9 – Component Mapping. (1) **PLC**. RTU is wrong because the description specifies factory floor, not remote telemetry; HMI does not drive I/O; SIS would not be the primary controller, only the protection layer. (2) **SIS**. A PLC or DCS could trip a valve but does not provide the SIL-rated independence required for "trips independent of the BPCS". Engineering WS and Historian have no I/O. (3) **Engineering Workstation**. The HMI does not download firmware, the historian does not hold project files, and the PLC itself does not hold the credentials – it accepts them. (4) **RTU**. PLCs are not designed for long unreliable links and do not buffer DNP3 events; SCADA Server sits at the master end, not the remote. (5) **Historian**. A PLC has no long-term storage; an HMI keeps short-term trends but not ten years; an Engineering WS is not the data serving system. (6) **HMI**. Historians provide raw data but not mimics; the SCADA Server can drive HMIs but the operator's physical viewing surface is the HMI; PLCs have no display.

Solution

Exercise 2.10 – Lab Notes (OpenPLC + Modbus TCP). Expected observations:

- OpenPLC Runtime listens on Modbus TCP port 502 by default.
- The HMI uses function code `0x01` or `0x02` to poll discrete inputs/coils (Start, Stop, Run), and `0x05` (write single coil) or `0x0F` (write multiple coils) to set Start/Stop. Holding-register access uses `0x03` read and `0x06` / `0x10` write.
- The traffic carries no authentication, no integrity check, no encryption – a Wireshark dissector shows full plaintext register addresses and values. Anyone on the VLAN can replay or forge a write to coil 0 (Start) and start the conveyor.
- The corrected ladder behaves identically to Exercise 2.8: brief Start press latches the Run output until Stop is asserted.